

Scheduling Algorithms Implementation Report

This report summarizes the changes made to the xv6 OS codebase to implement three scheduling algorithms: FCFS (First-Come, First-Served), Preemptive Priority, and Lottery Scheduling. The analysis covers all relevant files and highlights modifications, additions, and the overall impact on the kernel and user-level code.

1. FCFS (First-Come, First-Served) Scheduling

Changes:

- **proc.c / proc.h:** The scheduler logic was modified to select the process with the earliest arrival time. The process structure may include a field to track arrival time.
- **main.c / trap.c:** Context switch logic updated to ensure non-preemptive behavior, i.e., a running process is not interrupted until it yields or exits.
- **sysproc.c / syscall.c:** System calls related to process creation and termination may have been updated to support FCFS semantics.

Impact:

- Processes are scheduled strictly in the order they arrive.
- No process is preempted by another; CPU is allocated until voluntary yield or exit.

2. Preemptive Priority Scheduling

Changes:

- **proc.c / proc.h:** Added a **priority** field to the process structure. The scheduler now selects the highest-priority runnable process.
- **trap.c:** Timer interrupt handler modified to preempt the running process if a higher-priority process becomes runnable.
- **sysproc.c / syscall.c:** New system calls or arguments to set/get process priority.
- **user-level programs:** Utilities or test programs to demonstrate priority changes and preemption.

Impact:

- Processes with higher priority can preempt lower-priority ones.
- Dynamic priority changes are possible via system calls.
- Improved responsiveness for critical tasks.

3. Lottery Scheduling

Changes:

- **proc.c / proc.h:** Added a **tickets** field to the process structure. The scheduler randomly selects a runnable process based on ticket count.
- **random.c:** Implementation of a random number generator for lottery selection.
- **sysproc.c / syscall.c:** System calls to set/get the number of tickets for a process.

- **user-level programs:** Test programs to demonstrate probabilistic scheduling.

Impact:

- CPU time is distributed probabilistically according to ticket allocation.
- Fairness can be tuned by adjusting ticket counts.
- Demonstrates stochastic scheduling behavior.

General Kernel Modifications

- **defs.h / param.h:** Added or updated definitions for new fields and constants.
- **Makefile:** May include new source files (e.g., **random.c**) and user programs for testing.
- **usys.S / user.h:** Updated to support new system calls for priority and ticket management.

User-Level Changes

- New or modified user programs (e.g., **nice.c**, **ps.c**) to interact with the scheduler and demonstrate each algorithm.
- **myprog.c:** Custom program to spawn multiple processes with different priorities for Priority Scheduling, with different ticket counts for Lottery scheduling and observe scheduling order.

Benchmark Programs for Scheduling Comparison

To compare the performance of each scheduling algorithm, the following new user-level benchmark programs were added:

- **sched_bench_fcfs.c:** Measures turnaround time for multiple processes under FCFS scheduling.
- **sched_bench_priority.c:** Measures turnaround time for multiple processes with different priorities under Preemptive Priority scheduling.
- **sched_bench_lottery.c:** Measures turnaround time for multiple processes with different ticket allocations under Lottery scheduling.

Each program spawns several child processes with controlled workloads, records completion times, and prints total turnaround time. This allows direct comparison of scheduling efficiency and fairness.

Summary Table

Algorithm	Kernel Changes	User Changes	System Calls Added
FCFS	Scheduler, proc.c	Test programs	Possibly None
Preemptive Priority	Scheduler, trap.c	Priority utilities	Set/Get Priority
Lottery Scheduling	Scheduler, random.c	Ticket utilities	Set/Get Tickets

Conclusion

The codebase now supports three distinct scheduling algorithms, each with kernel and user-level changes. The modifications are modular, allowing for demonstration and testing of different scheduling strategies in the xv6 OS environment.